UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

INFORME TÉCNICO Y SUSTENTADO DEL PROYECTO

“Sistema de Gestión Académica Desarrollado Con React y Vite”

INTEGRANTES:

- QUISPE TICA Ruth Nayeli

DOCENTE:

- FERNANDEZ BEJARANO, Raul Enrique

CICLO: IV
SECCIÓN: A1

FECHA: 9 de noviembre del 2026

HUANCAYO - PERÚ
2026

1. INTRODUCCIÓN

El presente informe tiene como finalidad sustentar técnica y funcionalmente el desarrollo del proyecto denominado **Sistema de Gestión Académica**.

Este proyecto consiste en una aplicación web desarrollada para facilitar la administración de estudiantes, el registro de calificaciones y el seguimiento del rendimiento académico.

Su desarrollo responde a la necesidad de organizar la información académica mediante herramientas tecnológicas que permitan realizar operaciones de manera rápida, ordenada y sencilla.

Para su implementación se utilizó React como biblioteca principal para construir la interfaz, JavaScript para desarrollar la lógica de programación, Vite como herramienta de desarrollo y CSS para el diseño visual.

El sistema está dividido en tres módulos principales: Panel General, Gestión de Estudiantes y Gestión de Calificaciones. Cada módulo cumple una función específica dentro de la administración académica.

2. PLANTEAMIENTO DEL PROBLEMA

La administración de información académica requiere mantener organizados los datos personales de los estudiantes y sus respectivas calificaciones.

Cuando esta información se gestiona de manera manual o desorganizada, puede resultar difícil buscar registros, actualizar datos y realizar el seguimiento del rendimiento de cada estudiante.

Por esta razón, se desarrolló una aplicación web que centraliza estas operaciones en una sola interfaz, facilitando el acceso y la administración de la información.

3. OBJETIVOS

3.1. Objetivo general

Desarrollar un sistema web de gestión académica utilizando React y JavaScript, que permita administrar estudiantes, registrar calificaciones y visualizar el rendimiento académico.

3.2. Objetivos específicos

- Implementar un módulo para registrar, buscar, editar y eliminar estudiantes.
- Desarrollar un formulario con validaciones para evitar registros incompletos o duplicados.
- Implementar el registro de calificaciones en una escala de 0 a 20.

- Calcular automáticamente los promedios de los estudiantes.
- Clasificar el rendimiento académico mediante niveles.
- Mostrar estadísticas que faciliten el seguimiento académico.

4. FUNDAMENTO TECNOLÓGICO

Para el desarrollo del proyecto se utilizaron las siguientes tecnologías:

4.1. React

React es una biblioteca de JavaScript que permite construir interfaces de usuario mediante componentes reutilizables.

En el proyecto se utiliza para dividir la aplicación en elementos independientes. Por ejemplo, el archivo `App.jsx` organiza las vistas principales y administra la información de los estudiantes mediante el hook `useState`.

Sustento técnico: En `src/App.jsx` se utiliza:

```
const [students, setStudents] = useState(initialStudents)
```

Esta instrucción permite almacenar la información de los estudiantes en el estado de React y actualizarla cuando se registra, modifica o elimina un estudiante.

4.2. JavaScript

JavaScript es el lenguaje utilizado para desarrollar las operaciones y la lógica del sistema.

Se utiliza para:

- Registrar nuevos estudiantes.
- Actualizar información.
- Buscar registros.
- Eliminar estudiantes.
- Validar las calificaciones.
- Calcular promedios y clasificar resultados.

Sustento técnico: En `src/utils/grades.js` se encuentra la función `promedio()`, que utiliza `reduce()` para sumar las calificaciones y dividirlas entre la cantidad de notas registradas.

4.3. Vite

Vite es la herramienta que permite ejecutar el entorno de desarrollo de la aplicación.

Se utiliza para iniciar el servidor local y visualizar los cambios realizados durante la programación.

Sustento técnico: En el archivo `package.json` se encuentra configurado el comando:

`"dev": "vite"`

Esto permite iniciar el proyecto mediante `npm run dev`.

4.4. CSS3

CSS3 se utiliza para definir la presentación visual del sistema.

Permite establecer colores, tamaños, espacios, distribución de formularios, tablas, botones, menús y gráficos.

Sustento técnico: El archivo `src/styles.css` contiene las reglas de diseño que determinan la apariencia de los diferentes componentes.

4.5. Lucide React

Es una biblioteca de iconos utilizada para mejorar la presentación de la interfaz.

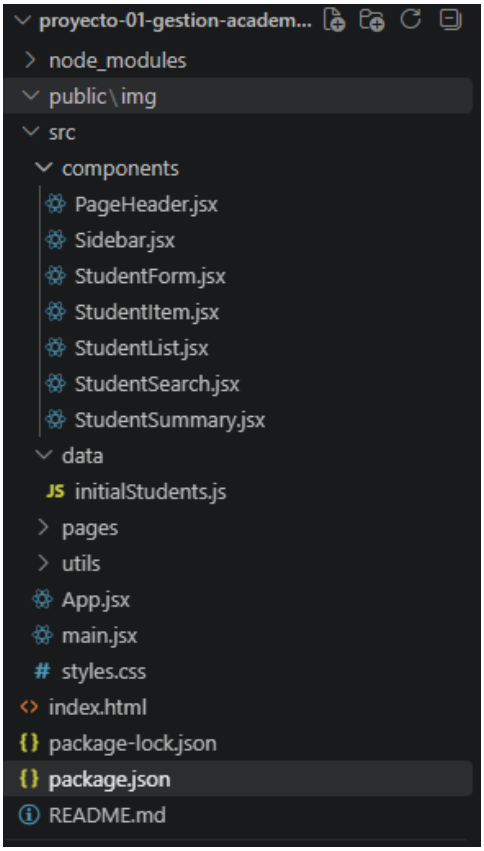
Se implementa mediante componentes como `Plus`, `Check`, `Pencil` y `Trash2`, que representan acciones como agregar, guardar, editar y eliminar.

5. ARQUITECTURA Y ORGANIZACIÓN DEL PROYECTO

El proyecto utiliza una organización basada en componentes, páginas, datos y funciones auxiliares.

La estructura principal es la siguiente:

- `src/App.jsx`: componente principal que controla las vistas y las operaciones generales.
- `src/main.jsx`: punto de entrada que inicia la aplicación React.
- `src/components/`: contiene los componentes reutilizables.
- `src/pages/`: contiene los módulos principales del sistema.
- `src/data/initialStudents.js`: contiene los datos iniciales y las carreras disponibles.
- `src/utils/grades.js`: contiene las funciones para calcular promedios y determinar niveles académicos.
- `src/styles.css`: contiene los estilos visuales.



Esta organización permite separar responsabilidades, facilitar el mantenimiento y evitar que toda la programación se encuentre concentrada en un solo archivo.

6. DESARROLLO Y SUSTENTACIÓN DE LOS MÓDULOS

6.1. Módulo de Panel General

<text>Archivo principal: `src/pages/Panel.jsx`.</text>

Este módulo presenta un resumen de la situación académica de los estudiantes.

Sus funciones son:

- Mostrar la cantidad de estudiantes registrados.
- Presentar la evolución del promedio grupal.
- Comparar los promedios por carrera.
- Identificar estudiantes con promedios inferiores a 12.
- Mostrar las actividades recientes.

Para identificar estudiantes que necesitan seguimiento se utiliza el método `filter()`, que permite seleccionar aquellos cuyo nivel académico es igual a `riesgo`.

También se utiliza `reduce()` para calcular el promedio grupal de cada evaluación.

El gráfico de evolución se construye mediante elementos SVG, que permiten representar visualmente los resultados.

Importancia: Este módulo permite interpretar los datos académicos y reconocer qué estudiantes necesitan mayor acompañamiento.

6.2. Módulo de Gestión de Estudiantes

<text>Archivo principal: `src/pages/Estudiantes.jsx`.</text>

Este módulo permite administrar los registros de los estudiantes.

Sus operaciones principales son:

1. Registrar nuevos estudiantes.
2. Buscar estudiantes por código o nombres.
3. Seleccionar un estudiante.
4. Modificar sus datos.
5. Eliminar registros.

El archivo `StudentForm.jsx` contiene el formulario utilizado para registrar y editar información.

Los campos solicitados son código, nombres, apellidos y carrera.

Se implementan validaciones para comprobar que los campos obligatorios estén completos y que el código no esté duplicado.

En `Estudiantes.jsx` se utiliza el método `filter()` para realizar las búsquedas y `find()` para localizar al estudiante seleccionado.

Para eliminar un registro se utiliza `filter()`, generando una nueva lista sin el estudiante eliminado.

Importancia: Este módulo permite mantener organizada la información de los estudiantes y facilita su administración.

6.3. Módulo de Gestión de Calificaciones

<text>Archivo principal: `src/pages/Calificaciones.jsx`.</text>

Este módulo permite registrar notas y consultar el rendimiento académico de cada estudiante.

Sus funcionalidades son:

- Seleccionar un estudiante.
- Registrar una calificación.
- Validar que la nota esté entre 0 y 20.
- Calcular automáticamente el promedio.
- Mostrar el nivel de rendimiento.
- Presentar la distribución de resultados.

El archivo `src/utils/grades.js` contiene las funciones responsables de los cálculos académicos.

La función `promedio()` suma las notas mediante `reduce()` y divide el resultado entre la cantidad de calificaciones.

La función `nivel()` determina la clasificación académica utilizando condiciones:

- De 18 a 20: Destacado.
- De 12 a menos de 18: Aprobado.
- Menor de 12: En riesgo.
- Sin calificaciones: Sin notas.

En `Calificaciones.jsx` también se valida que el usuario no registre valores inferiores a 0 o superiores a 20.

Importancia: Este módulo facilita el control de las calificaciones y permite identificar el rendimiento académico de cada estudiante.

7. EXPLICACIÓN DE LAS PRINCIPALES FUNCIONES DE PROGRAMACIÓN

Durante el desarrollo se utilizaron diferentes métodos y herramientas de JavaScript.

Método o herramienta	Aplicación en el proyecto
`useState()`	Almacena y actualiza los datos de la aplicación.
`useEffect()`	Sincroniza el formulario con el estudiante seleccionado.
`useMemo()`	Optimiza el cálculo de la lista filtrada de estudiantes.
`map()`	Recorre los registros y genera elementos actualizados.
`filter()`	Busca o elimina elementos de una lista.
`find()`	Encuentra un estudiante específico.
`reduce()`	Calcula la suma y los promedios.
`some()`	Comprueba si existe un código duplicado.

Estos métodos permiten desarrollar operaciones dinámicas sin necesidad de modificar manualmente cada registro.

8. FUNCIONAMIENTO GENERAL DEL SISTEMA

El funcionamiento se desarrolla mediante el siguiente procedimiento:

1. El usuario inicia la aplicación en el navegador.
2. El sistema muestra el Panel General.
3. El usuario ingresa al módulo de estudiantes.
4. Registra los datos personales y selecciona la carrera.
5. El sistema valida la información ingresada.
6. El usuario accede al módulo de calificaciones.

7. Selecciona al estudiante y registra una nota.
8. El sistema comprueba que la calificación sea válida.
9. Se actualiza la información y se calcula el promedio.
10. El Panel General presenta las estadísticas correspondientes.

De esta manera, los tres módulos trabajan con la misma información administrada desde el componente principal `App.jsx`.

9. INSTALACIÓN Y EJECUCIÓN

Para ejecutar el proyecto se necesita instalar Node.js, contar con Visual Studio Code y disponer de la carpeta del proyecto.

Procedimiento:

1. Descomprimir el proyecto.
2. Abrir la carpeta en Visual Studio Code.
3. Abrir la terminal integrada.
4. Ejecutar:

```
npm install
```

Este comando instala las dependencias declaradas en `package.json`, como React, React DOM, Vite y Lucide React.

5. Iniciar el servidor de desarrollo:

```
npm run dev
```

6. Abrir en el navegador la dirección local que muestra la terminal.
7. Para comprobar que el proyecto puede compilarse para producción, ejecutar:

```
npm run build
```

Este último comando genera una versión optimizada de la aplicación.

10. PRUEBAS Y VALIDACIONES DEL SISTEMA

Para comprobar el funcionamiento se consideran las siguientes pruebas:

Prueba	Resultado esperado
Registrar un estudiante con todos sus datos	El registro se agrega correctamente.
Registrar un código duplicado	El sistema muestra un mensaje de error.
Dejar campos obligatorios vacíos	El formulario solicita completar los datos.
Buscar por nombre o código	Se muestran los resultados coincidentes.
Editar un estudiante	Se actualiza su información.
Eliminar un estudiante	El registro desaparece de la lista.
Registrar una nota entre 0 y 20	La calificación se agrega correctamente.

Registrar una nota superior a 20	El sistema rechaza el valor.
Consultar el promedio	Se muestra el resultado calculado.

Estas pruebas permiten comprobar las validaciones y las operaciones implementadas en el código.

11. LIMITACIONES Y MEJORAS FUTURAS

Una característica importante del proyecto es que actualmente utiliza el estado de React para administrar los datos en memoria.

Esto significa que los registros nuevos y las calificaciones agregadas durante la ejecución no se conservan después de recargar la página.

Además, el proyecto todavía no incorpora una base de datos ni un sistema de autenticación.

Como mejoras futuras se propone:

- Implementar una base de datos MySQL o PostgreSQL.
- Crear un sistema de inicio de sesión.
- Incorporar diferentes roles de usuario.
- Generar reportes académicos en PDF.
- Permitir exportar información a Excel.
- Implementar copias de seguridad de los registros.

12. CONCLUSIONES

1. Se desarrolló una aplicación web para administrar estudiantes y controlar sus calificaciones utilizando React y JavaScript.
2. La separación del sistema en componentes y módulos permitió organizar mejor el código y facilitar su mantenimiento.
3. La implementación de validaciones permite evitar registros incompletos, códigos duplicados y calificaciones fuera del rango establecido.
4. Las funciones de cálculo permiten obtener promedios y clasificar el rendimiento académico de manera automática.
5. El proyecto demuestra la aplicación práctica de herramientas de desarrollo web, aunque requiere implementar una base de datos para garantizar la conservación permanente de la información.

13. REFERENCIAS TÉCNICAS

- Documentación oficial de React: https://react.dev/
- Documentación oficial de Vite: https://vite.dev/
- Documentación de JavaScript: https://developer.mozilla.org/es/docs/Web/JavaScript
- Documentación de Lucide React: https://lucide.dev/guide/packages/lucide-react